



COMP2017 9017

Assignment 2

Due: 23:59:00 20 May 2026

This assignment is worth 12.5% of your final assessment

This assessment is CONFIDENTIAL. © University of Sydney

Contents

1	Assignment 2- [PointerPro Animate Service] - 12.5%	3
2	Introduction	6
3	Task	6
4	Communication protocol	7
4.1	Connection	7
4.2	Authorisation	8
4.2.1	users.txt Format	8
4.3	Animate remote procedure call protocol	9
4.4	Protocol Exceptions	10
4.5	Disconnect protocol	11
5	Starvation	11
5.1	[COMP9017 only]	12
6	Marking	12
7	Short Answer Questions	12
8	Restrictions	13
9	References	14
10	Submission Checklist	14

A	Version history	15
B	API documentation	15

List of Acronyms

API Application Programming Interface. [9](#), [10](#)

ID Identifier. [6](#), [10](#), [11](#)

PID Process Identifier. [7](#)

RPC Remote Procedure Call. [6](#), [8](#), [9](#), [10](#), [11](#), [12](#)

UI User Interface. [11](#)

1 Assignment 2- [PointerPro Animate Service] - 12.5%

We strongly recommend reading this entire document at least twice. You are encouraged to ask questions on Ed after you have first **searched**, and checked for updates of this document. If the question has not been asked before, make sure your question post is of type "**Question**" and is under "**Assignment**" category → "**P2**". Please follow the staff directions for using the question template.

It is important that you continually back up your assignment files onto your own machine, flash drives, external hard drives and cloud storage providers (as private). You are encouraged to submit your assignment regularly while you are in the process of completing it.

Full reproduction steps (seed, description of what you tried) **MUST** be given if you are enquiring about a test failure or if you believe there is a bug in the marking script.

Academic Declaration

By submitting this assignment you declare the following: I declare that I have read and understood the University of Sydney Student Plagiarism: Academic Integrity Policy, Coursework Policy and Procedures, and except where specifically acknowledged, the work contained in this assignment/project is my own work, and has not been copied from other sources or been previously submitted for award or assessment.

I understand that failure to comply with the Student Plagiarism: Academic Integrity Policy, Coursework Policy and Procedures can lead to severe penalties as outlined under Chapter 8 of the University of Sydney By-Law 1999 (as amended). These penalties may be imposed in cases where any significant portion of my submitted work has been copied without proper acknowledgement from other sources, including published works, the Internet, Generative AI where approved, existing programs, the work of other students, or work previously submitted for other awards or assessments.

I acknowledge that I have reviewed and understood the University of Sydney's guidelines on the responsible use of Generative AI ¹ and will adhere to them in accordance with academic integrity policies.

I realise that I may be asked to identify those portions of the work contributed by me and required to demonstrate my knowledge of the relevant material by answering oral questions or by undertaking supplementary work, either written or in the tutorial, in order to arrive at the final assessment mark.

I acknowledge that the School of Computer Science, in assessing this assignment, may reproduce it entirely, may provide a copy to another member of faculty, and/or communicate a copy of this assignment to a plagiarism checking service or in-house computer program, and that a copy of the assignment may be maintained by the service or the School of Computer Science for the purpose of future plagiarism checking.

This assignment will be submitted in Ed Lessons, under **Assignments - P2**.

In addition to the Academic Integrity and Compliance statements below, an Academic Declaration appears in your assignment in Ed. All statements apply.

¹<https://www.sydney.edu.au/students/academic-integrity/artificial-intelligence.html>

Academic Integrity

While the University is aware that the vast majority of students and staff act ethically and honestly, it is opposed to and will not tolerate academic integrity breaches and will treat all allegations seriously.

Further information on academic integrity, and the resources available to all students can be found on the academic integrity pages on the current students website:

<https://sydney.edu.au/students/academic-integrity.html>

You may only use generative artificial intelligence (AI) and automated writing tools in assessment tasks if you are permitted to by your unit coordinator. If you do use these tools, you must acknowledge this in your work, either in a footnote or an acknowledgement section. For information on acknowledging AI please refer to the guidance in the [AI in Education Canvas site](#).

We use Turnitin, which includes AI detection, to detect potential instances of plagiarism or other forms of academic integrity breach. If such matches indicate evidence of plagiarism or other forms of academic integrity breaches, your teacher is required to report your work for further investigation.

Further information on research integrity and ethics for postgraduate research students and students undertaking research-focused coursework such as Honours and capstone research projects can also be found on the current students website:

<https://sydney.edu.au/students/research-integrity-ethics.html>

Compliance Statement

In submitting this work, I acknowledge I have understood the following:

1. I have read and understood the University of Sydney's [Academic Integrity Policy](#).
2. The work is substantially my own and, where any parts of this work are not my own, I have indicated this by acknowledging the source of those parts of the work and clearly indicated any quoted text by quotation marks or indentation according to accepted style standards.
3. I have acknowledged any assistance provided in preparing the work including the use of copy-editing, proof-reading, and automated writing and drawing tools (including artificial intelligence (AI), reference generators, translation software, grammar checkers, but not spell checkers).
4. The work has not previously been submitted in part or in full for assessment in another unit unless I have been given permission by my unit of study coordinator to do so.
5. The work will be submitted to similarity detection software (Turnitin) and a copy of the work will be retained in Turnitin's paper repository for future similarity checking.
6. Engaging in plagiarism or academic dishonesty in coursework will, if detected, lead to the University commencing proceedings under the [Academic Integrity Policy](#) and the [Academic Integrity Procedures](#).
7. Engaging in plagiarism or academic dishonesty in research-focused work will lead to the University commencing proceedings under the [Research Code of Conduct](#) and the [Academic Integrity Procedures](#).

8. Engaging another person to complete part or all of the submitted work will, if detected, lead to the University commencing proceedings against me for potential student misconduct under the [University of Sydney \(Student Discipline\) Rule](#).
9. That the School of Computer Science, in assessing this assignment, may reproduce it entirely, may provide a copy to another member of faculty, and/or communicate a copy of this assignment to a plagiarism checking service or in-house computer program, and that a copy of the assignment may be maintained by the service or the School of Computer Science for the purpose of future plagiarism checking.

You are advised to keep copies of your assignment submission, drafts and any other research materials as evidence of your research and writing process. If you have used AI in the completion of your assignment, you should keep copies of the AI outputs.

2 Introduction

Memory Leak Studios's PointerPro Animate has been an overwhelming success, but users are expressing a need for multiple clients to collaborate on a project. In order to support this, the Animate service must accept connections and directives from multiple clients.

Collaboration here means operating on the same canvas. Users may share their canvas [Identifiers \(IDs\)](#) with other users in order to collaborate, or they may operate on their own private canvas. When operating on a shared canvas, it is important that operations by each individual user occur in the order in which they are issued. However, there is no need to guarantee ordering across clients. For shared canvases, it is up to users to synchronise their work. Users must issue an explicit *BARRIER* operation, for which future operations will not be applied by that user until all other users of that canvas have also issued a *BARRIER* operation.

3 Task

Your task is to write both a client program and a server program that can be used to facilitate collaborative work between multiple clients. Your server will make use of the existing *PointerPro animate* software – it has been provided in the form of a static library that your server should link against (but your client should not). In order to make use of a high performance machine that is used to host the server, your server must provide parallel execution of requests where possible. The behaviour of client and server is as follows:

1. **server** — the process managing canvases, operations on those canvases and synchronisation between clients.
 - Listens for client connections using signals.
 - Creates named pipes (FIFOs)² for bidirectional communication between itself and each client.
 - Applies animate [Remote Procedure Calls \(RPCs\)](#) received on those FIFOs in the appropriate order.
 - Generates frame data and animations in mp4 format when requested.
 - Handles graceful disconnects from clients and error reporting.
2. **client** — a per-user process that interacts with the server.
 - Establishes a service connection using signals
 - Sends user commands of the prescribed format via the FIFOs that were created by the server.
 - Sends graceful disconnect messages to the server on termination.

²See `man 3 mkfifo`.

Deliverables & Constraints

- Makefile and source code for generating two executable programs named *animate_server* and *animate_client*. Note that source code for the animate library is not required to be provided.
- Must compile on Edstem Linux environment with no warnings (using `-Wall -Wextra`).

4 Communication protocol

This section defines the communication protocol that must be followed between client and server. Note that all messages sent from client to server and server to client should contain exactly one ‘\n’ character at the end of the message.

4.1 Connection

1. Start the server by running the *animate_server* binary and specifying a limit, of at least 1, on the number of threads that will interact with libanimate and *ffmpeg*:
`./animate_server <threadpool size>`
While there is no limit on the number of other threads in your system, the total number of threads must be constant after initialisation (i.e. not be dependent on the number of clients).
2. The server starts and prints its **Process Identifier (PID)** to *stdout*. Clients must be informed of the Server **PID** in order to register itself as a client.
Server PID: <server pid>.
3. Clients can then be started by running the *animate_client* binary and specifying the **PID** reported by the server:
`./animate_client <server pid>`
4. At startup, clients immediately send the SIGUSR1 signal to the server’s **PID** and then block until the server responds by sending SIGUSR2. A client may timeout after approximately 1 second if no signal is received. Therefore, a response to SIGUSR1 must be immediate.
5. On receiving SIGUSR1, the server must:
 - 5.1. Create two named FIFOs to establish bi-directional communication:
 - FIFO_C2S_<client_pid> (Client-to-Server)
 - FIFO_S2C_<client_pid> (Server-to-Client)
 - If either already exist, unlink them before creating new FIFOs.
 - 5.2. Send SIGUSR2 back to the client to indicate completion.
6. The client, upon receiving SIGUSR2, opens:
 - FIFO_C2S_<client_pid> for writing, and
 - FIFO_S2C_<client_pid> for reading,

4.2 Authorisation

Once the client has registered a connection with the server, the client waits for user input on *stdin*. User input can be relayed directly to the server as the [RPC](#) protocols match. However, the client should refrain from sending animate specific [RPC](#) commands until authorisation has been established. The client should respond to such user requests with the text “Not logged in”. The following procedure outlines the process of logging in and checking authorisation.

1. When the user enters: “Login <username>\n”, the client relays this message to the server via `FIFO_C2S_<client_pid>`. Usernames will be discussed further in [Section 4.2.1](#).
2. After receiving the client’s username the server must:
 - 2.1. Check the provided username against the `users.txt` file ([Section 4.2.1](#)) to determine if the user is authorised and if their account has a balance greater than 0.
 - 2.2. If the username is found and is associated with a balance greater than 0, the server must respond as follows by writing to the clients `FIFO_S2C_<client_pid>`:
 - The server must send the user’s current balance as a human-readable signed integer: “<Balance>”.
 - The client should report to the user via *stdout*:
“Welcome <username>. Your balance is <balance>”
 - 2.3. If the username is found but is associated with a balance less than or equal to 0:
 - The server must send a rejection message:
“Reject BALANCE”.
 - The client should relay this message to the user via *stdout*.
 - 2.4. If the username is not found:
 - The server must send a rejection message:
“Reject UNAUTHORISED”.
 - The client should relay this message to the user via *stdout*.
 - 2.5. If a rejection message was sent:
 - The server should wait for approximately 1 second (without blocking the whole process) before disconnecting from the client by closing both FIFOs and unlinking them (i.e. delete them from the file system).

4.2.1 users.txt Format

The `users.txt` file must reside in the same directory as the running server executable. Each line in the file contains a username followed by an account balance, separated by one or more spaces or tab characters. The username is an alphanumeric character sequence of at most 32 characters. The balance is a signed integer. The `users.txt` file is updated monthly while the server is down for maintenance.

Example format:

1	ExcitableFabricator	1
2	UnwillingDeveloper	10
3	QueasyMidwife	0
4	JollyPainter	-10

Leading and trailing whitespace around usernames and balance must be ignored. Usernames are case-sensitive and contain no spaces.

4.3 Animate remote procedure call protocol

Interaction between clients and the Animate [Application Programming Interface \(API\)](#) on the server are expressed as a human readable strings. These strings communicate the function name (without the animate prefix), followed by the argument values as decimal integers or strings. Decimal integers will be signed or unsigned depending on the argument type. For example, to create a bitmap sprite and rectangle sprite:

```
"create_sprite rick.bmp"
"create_rectangle 10 10 12345 1"
```

Recall that all messages sent from client to server and server to client should contain exactly one ‘\n’ character at the end of the message. This character will not be shown in the messages presented in this section.

When developing the animate library, you were able to assume that function call parameters were valid. In this case, we cannot be sure who wrote the code that is communicating with the server. Since this input can no longer be trusted, it is important that all parameters are checked for correctness and return an error code. The first token returned by an [RPC](#) call will indicate the success (0), (-1) if the [RPC](#) call could not be interpreted or (-2) if a parameter value was invalid or (-3) for any other error related to the server e.g. out of memory. If failure is indicated, there is no need to send any additional information back to the client.

If the [RPC](#) was successful and the remote function returns a value, the [RPC](#) result token should be followed by the return value of the remote function.

The server therefore sends one of the following messages in response to an [RPC](#) and the corresponding text is printed for the user:

Cause	Server -> Client	Displayed to user
The RPC call failed	"-1"	"RPC Failed"
Error in value	"-2"	"Value error"
Internal error	"-3"	"Internal error"
Success	"0"	"Success"
Success	"0 <API function return value>"	"Success <value>"

Functions that return boolean values should have the return value sent back to the client as an integer value (0 for false, 1 for true).

The animate [API](#) functions corresponding to the examples above return a memory address that does not correspond to a valid address in the client. Recall that we have chosen abstract data types – we

can be sure that the client does not depend on the content at these addresses. You are free to pass the memory address directly as a handle to the object created, or you may like to define your own number space for such handles. In either case, the value returned and used in subsequent calls will be a `uint64_t` and the value 0 is reserved to indicate an invalid handle. The server must verify that the handle is valid and owned by (or shared with) the requesting client before use.

Note that clients need not wait for the return value of one **RPCs** before sending the next **RPC**. These requests must be processed in parallel, where possible. However, the server must ensure that response messages are sent in the same order in which their corresponding request arrived.

4.4 Protocol Exceptions

The following **API** functions are not supported over **RPC**:

1. `animate_set_animation_function`
2. `animate_frame_size_bytes`
3. `animate_generate_frame`

The following additional **RPCs** must be supported:

1. “generate <canvas> <filename> <start> <end> <frame rate>”

This **RPC** will generate frames starting at index <start> and ending at <end> inclusive, using a frame rate of <frame rate> for the canvas with **ID** <canvas>. The **RPC** will result in 3 files being written:

- (a) The resulting frame data must be saved in a data file named <filename>.dat.
- (b) The external program *ffmpeg* must be used to generate the file <filename>.mp4 using the resulting frame data.
- (c) The *stdout* and *stderr* output of *ffmpeg* must be saved to the file <filename>.log.

There are six possible reply messages for this **RPC**:

Cause	Server -> Client	Displayed to user
The RPC call failed	“-1”	“RPC Failed”
Error in value	“-2”	“Value error”
Internal error	“-3”	“Interval error”
Writing <filename>.dat failed	“0 -1”	“Data write failed”
Writing <filename>.mp4 failed	“0 0 -1”	“Movie write failed”
Writing <filename>.log failed	“0 0 -1”	“Movie write failed”
Success	“0 0 0”	“Success”

Note that the response for failing to write the log file and failing to write the movie file are intentionally equivalent.

Note also that this **API** call does not modify the underlying canvas. These requests can therefore be processed in parallel, even when the same canvas is used.

2. “share_canvas <canvas> <username>”

A canvas can be shared by multiple clients. This **RPC** is used by both the client who created the canvas with **ID** canvas and the client who wishes to use the canvas with **ID** <canvas>. In each case, <username> must correspond to the username of the other client. It is up to the clients to communicate canvas handles between themselves. This **RPC** can lead to the following responses from the server:

Cause	Server -> Client	Displayed to user
The RPC call failed	“-1”	“RPC Failed”
Error in value	“-2”	“Value error”
Internal error	“-3”	“Internal error”
Success	“0”	“Success”

3. “barrier <canvas>”

With multiple clients sharing a canvas, they may require synchronisation when the order of operations is important. In this case, all clients with whom the canvas **ID** <canvas> is shared must issue this barrier **RPC** to ensure that all operations on that canvas have been completed before issuing more **RPC** on this canvas. The response should only be sent once all non-terminated clients sharing the canvas have issued the barrier **RPC**. This **RPC** can lead to the following responses from the server:

Cause	Server -> Client	Displayed to user
The RPC call failed	“-1”	“RPC Failed”
Error in value	“-2”	“Value error”
Internal error	“-3”	“Internal error”
Success	“0”	“Success”

4.5 Disconnect protocol

A client gracefully disconnects using the following **RPC**:
“Disconnect”

However, **User Interface (UI)** designers are more concerned with productivity over robustness. They expect users to tolerate occasional failures. You are required to periodically check if the client process has terminated without grace and react as if the client has sent a **Disconnect** message.

On disconnect, the server should free any resources used by only the disconnecting client. Resources used by canvases that have been shared with clients that are still connected should remain available.

5 Starvation

The size of the thread pool – the number of threads allowed – may be far fewer than the number of clients. It is important that each thread created is not restricted to serving a limited set of clients. Each thread should serve requests that arrive from any client and best effort should be made to ensure one client does not receive priority over others.

Clients need not block waiting for the return value of one **RPCs** before sending the next **RPC**. These requests must be processed in parallel, where possible. However, the server must ensure that response messages are sent in the same order in which their corresponding request arrived.

5.1 [COMP9017 only]

ffmpeg can take quite some time to execute. If the number of movie files being processed is equal to the size of the thread pool, simple [RPC](#) calls may face significant delay as all threads in the thread pool are blocked waiting for *ffmpeg* to complete.

The additional task for COMP9017 students is to allow these threads to continue processing other requests rather than waiting for *ffmpeg* to complete.

Recall that server responses to clients should be delivered in the order that their corresponding requests are received.

6 Marking

Animate has already been thoroughly tested. Therefore, marking will focus on client-server interoperability testing, parallelism, concurrency and adherence to the protocol specification.

On submission, simple testing will be conducted on your client and server independently to ensure conformity. However, the test suite is incomplete and does not reflect the overall mark that you might receive. You are expected to conduct your own tests, ideally using automated test scripts or programs.

For style, refer to the [style guide](#). You will also be marked based on the modularity and organisation of your code. For full marks, code should be organised in multiple source files, and use modular, task-specific functions. Organised data structures are essential here. Style marking is only applied for reasonable attempts at correctness.

7 Short Answer Questions

As part of the in-tutorial code interview in week 13, you are required to analyse your code and prepare to answer two of the below questions. You must supplement your answer with references to your code. The examiner will also ask follow-up questions based on your response. **COMP9017** students must answer [Q4](#).

- Q1 How do you ensure that the canvas/sprite/placement ID provided by the user during an RPC call is valid, and validate that the user has access to use it?
- Q2 How do you ensure that a canvas is only destroyed when all users who share the canvas are disconnected?
- Q3 How does your program handle multiple simultaneous users? What other approaches have you considered, and what would be the trade-offs?
- Q4 How did you (or would you) test 1 client and multiple clients interacting with the server simultaneously?

8 Restrictions

To successfully complete this assignment you *must* follow the restrictions below. Submissions breaking these restrictions will receive a deduction of up to 1 mark *per breach*.

Submissions with any attempt to deceive or disrupt the marking system will result in an immediate zero.

- The implementation of the server must not create more threads than allowed (see Section 4.1).
- The code must entirely be written in the C programming language.
- Free all dynamic memory that is used.
- POSIX library is needed for POSIX threads, however other libraries, such as `regex.h`, are *NOT* allowed.
- *NOT* use any external libraries other than those in `libc`.
- *NOT* use VLAs.
- *NOT* use shared memory.
- *NOT* use `regex`
- *NOT* have unclean repositories. This means no object, executable, or temporary files **for any commit** in the latest repository. A clean final commit is *NOT* good enough. You **SHOULD** use `.gitignore` to avoid committing garbage files.
- Only include header files that contain declarations of functions and extern variables. Do not define functions within header files.
- Any and all comments, including git commit messages, must be written only in the English language.
- Must use meaningful commits and meaningful comments on commits. ³
- *NOT* manually use return code 42, reserved by ASAN.
- Other restricted functions may come at a later date.

³"forcing the seed of a test case" does not count as valid commit. Must cite reason and identified failure.

9 References

AI is allowed, as long as the majority of the work is your own. However, you must include relevant information about your use of AI such that it could be reproduced. This includes:

- Version, Company, Date
- Public URL
- Prompt history (saved as text, not images, graphics or screenshots)

All resources relating to your references should be placed in your submission under the directory `./references/`.

Please follow the below suggested format for ease of viewing. Each conversation should be saved as a separate file.

```
1 Company: <company, or local_hosted>
2 Model: <model>
3 Date: <dd/mm/yyyy>
4
5 Public URL: <url_here>
6
7 Prompt History:
8 <
9     prompt history in text format.
10    Include the prompt that you used as text, then the response given.
11 >
12
13 Use case: <brief_description>
```

You should also include in your `./references/`:

- `summary.txt` - Give a summary of how AI was used: which code, what tests, test driver or other.

10 Submission Checklist

- ☐ Submission has a valid makefile with the rule for targets:
`animate_server` and `animate_client`, compiled using `make <target>`.
- ☐ Program is organised into multiple source and header files (for larger programs).
- ☐ Not include any object file, binary, or junk data in your git repo.
- ☐ If you have used AI, a directory references containing files as detailed in Section 9.

Any memory leaks result in a 30% deduction after all other marks have been calculated, but will not decrease your mark below 40%.

A Version history

We aim to resolve all spec updates within the first 3-5 days.

30/04/2026

- Clarified how `animate_client` should be started

3/05/2026

- For `sprite/placement/canvas` handles, 0 shall be reserved to indicate invalid values, regardless of the number system used for handles.
- Added RPC error code -3 to catch any other failure on the server side.
- Clarified client behaviour when not logged in – client prints “Not logged in”.
- Clarified that there is no limit on the total number of threads in the system, but it must not depend on the number of clients, and only threadpool threads can interact with `libanimate` or use `ffmpeg`.
- Clarified response to failing to write log files – same as failing to write movie files.
- fixed typo “server relays this message to the server”

13/05/2026

- Fixed typo - `interval->internal`
- Fixed contradiction in restrictions – referred to relevant section instead.

B API documentation

The API documentation has been attached on the following pages for your convenience.

PointerPro Animate

Generated by Doxygen 1.9.8

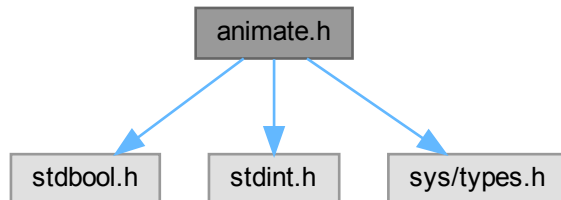
Chapter 1

File Documentation

1.1 animate.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for animate.h:



Typedefs

- typedef uint32_t [color_t](#)
Representation of a pixel.
- typedef void(* [animate_fn](#)) (void *priv, ssize_t *x, ssize_t *y, float t)
Custom animation function for optional extension.

Functions

- struct canvas * [animate_create_canvas](#) (size_t height, size_t width, [color_t](#) background_color)
Create the canvas on which our animation will be built.
- void [animate_destroy_canvas](#) (struct canvas *canvas)
Clean up a canvas.
- struct sprite * [animate_create_sprite](#) (const char *file)

- Create a sprite from the provided input bitmap file.*
 - struct sprite * [animate_create_rectangle](#) (size_t width, size_t height, [color_t](#) c, bool filled)
- Create a sprite that represents a rectangle.*
 - struct sprite * [animate_create_circle](#) (size_t radius, [color_t](#) c, bool filled)
- Create a sprite that represents a circle.*
 - bool [animate_destroy_sprite](#) (struct sprite *sprite)
- Frees all memory associated with a sprite.*
 - struct sprite_placement * [animate_place_sprite](#) (struct canvas *canvas, struct sprite *sprite, ssize_t x, ssize_t y)
- Places a sprite on the top layer of the canvas.*
 - void [animate_placement_up](#) (struct sprite_placement *sprite_placement)
- Raise a sprite one level towards the top layer.*
 - void [animate_placement_down](#) (struct sprite_placement *sprite_placement)
- Lower a sprite one level towards the bottom layer.*
 - void [animate_placement_top](#) (struct sprite_placement *sprite_placement)
- Raise the sprite to the top-most layer.*
 - void [animate_placement_bottom](#) (struct sprite_placement *sprite_placement)
- Lower the sprite to the bottom-most layer.*
 - void [animate_destroy_placement](#) (struct sprite_placement *sprite_placement)
- Removes the placement from its associated canvas and free resources.*
 - void [animate_set_animation_params](#) (struct sprite_placement *sprite_placement, ssize_t vx, ssize_t vy, ssize_t ax, ssize_t ay)
- Configure sprite animation.*
 - void [animate_set_animation_function](#) (struct sprite_placement *sprite_placement, [animate_fn](#), void *priv)
- size_t [animate_frame_size_bytes](#) (struct canvas *canvas)
- Return the size in bytes of an animation frame.*
 - void [animate_generate_frame](#) (const struct canvas *canvas, size_t frame, size_t frame_rate, void *buf)
- Generate a frame of the animation.*

1.1.1 Typedef Documentation

1.1.1.1 animate_fn

```
typedef void(* animate_fn) (void *priv, ssize_t *x, ssize_t *y, float t)
```

Custom animation function for optional extension.

Parameters

<i>priv</i>	a handle to the private data provided when the function was registered
<i>x</i>	a reference to the starting position. On return, this should contain the target sprite position for the frame in question.
<i>y</i>	a reference to the starting position. On return, this should contain the position for the frame in question.
<i>t</i>	the current time since the start of the animation.

1.1.1.2 color_t

```
typedef uint32_t color_t
```

Representation of a pixel.

1.1.2 Function Documentation

1.1.2.1 animate_create_canvas()

```
struct canvas * animate_create_canvas (
    size_t height,
    size_t width,
    color_t background_color )
```

Create the canvas on which our animation will be built.

Parameters

<i>height</i>	height of the drawing area in pixels
<i>width</i>	width of the drawing area in pixels
<i>background_color</i>	the colour to use when no sprites overlap a pixel.

Returns

Abstract data type for the canvas. The memory associated this struct is allocated by this function and cleaned up by the corresponding destroy function

1.1.2.2 animate_create_circle()

```
struct sprite * animate_create_circle (
    size_t radius,
    color_t c,
    bool filled )
```

Create a sprite that represents a circle.

We currently only support filled circles. The circle pixels can be determined by the following formula: $x*x + y*y \leq r*r$ Where x is the x offset from the center of the circle y is the y offset from the center of the circle r is the radius of the circle

Parameters

<i>radius</i>	the radius of the circle
<i>c</i>	the colour of the circle
<i>filled</i>	reserved for future use

Returns

A handle to the created sprite

1.1.2.3 animate_create_rectangle()

```
struct sprite * animate_create_rectangle (
    size_t width,
```

```
size_t height,
color_t c,
bool filled )
```

Create a sprite that represents a rectangle.

Parameters

<i>width</i>	the width of the rectangle
<i>height</i>	the height of the rectangle
<i>c</i>	the colour of the rectangle
<i>filled</i>	true if the rectangle should be filled, otherwise false

Returns

A handle to the created sprite

1.1.2.4 animate_create_sprite()

```
struct sprite * animate_create_sprite (
    const char * file )
```

Create a sprite from the provided input bitmap file.

The file format can be assumed to be ARGB32. The bitmap content may be loaded only on first access, however it must only be loaded once. NOTE: the order of image rows differs to what we expect. The first pixel in the bmp file format is on the bottom left while the last pixel is the top right. For simplicity, reverse the order of rows while loading the bitmap.

Parameters

<i>file</i>	the name of the bitmap file to be loaded
-------------	--

1.1.2.5 animate_destroy_canvas()

```
void animate_destroy_canvas (
    struct canvas * canvas )
```

Clean up a canvas.

Frees all memory associated with the canvas, including placements, but not sprites

Parameters

<i>canvas</i>	the canvas to clean up
---------------	------------------------

1.1.2.6 animate_destroy_placement()

```
void animate_destroy_placement (
    struct sprite_placement * sprite_placement )
```

Removes the placement from its associated canvas and free resources.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.7 animate_destroy_sprite()

```
bool animate_destroy_sprite (
    struct sprite * sprite )
```

Frees all memory associated with a sprite.

Parameters

<i>sprite</i>	the sprite to clean up
---------------	------------------------

Returns

0 on success, otherwise an error occurred. The only error possible is that the sprite is still in use

1.1.2.8 animate_frame_size_bytes()

```
size_t animate_frame_size_bytes (
    struct canvas * canvas )
```

Return the size in bytes of an animation frame.

This is useful for the caller to determine how much memory to allocate when combining sprites into a frame.

Parameters

<i>canvas</i>	the canvas in question
---------------	------------------------

Returns

the size, in bytes, of a frame for this canvas

1.1.2.9 animate_generate_frame()

```
void animate_generate_frame (
    const struct canvas * canvas,
```

```

size_t frame,
size_t frame_rate,
void * buf )

```

Generate a frame of the animation.

Parameters

<i>canvas</i>	the canvas describing the animation
<i>frame</i>	the frame number (starting from 0)
<i>frame_rate</i>	the number of frames per second in the animation
<i>buf</i>	a buffer provided by the user that is large enough to hold a frame. This buffer will contain the frame data on return.

1.1.2.10 `animate_place_sprite()`

```

struct sprite_placement * animate_place_sprite (
    struct canvas * canvas,
    struct sprite * sprite,
    ssize_t x,
    ssize_t y )

```

Places a sprite on the top layer of the canvas.

Parameters

<i>canvas</i>	the canvas to place the sprite on
<i>sprite</i>	the sprite to place
<i>x</i>	the x coordinate of the canvas to which the top left pixel of the sprite should be placed
<i>y</i>	the y coordinate of the canvas to which the top left pixel of the sprite should be placed

Returns

A handle to a sprite placement.

1.1.2.11 `animate_placement_bottom()`

```

void animate_placement_bottom (
    struct sprite_placement * sprite_placement )

```

Lower the sprite to the bottom-most layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.12 `animate_placement_down()`

```
void animate_placement_down (
    struct sprite_placement * sprite_placement )
```

Lower a sprite one level towards the bottom layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.13 `animate_placement_top()`

```
void animate_placement_top (
    struct sprite_placement * sprite_placement )
```

Raise the sprite to the top-most layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.14 `animate_placement_up()`

```
void animate_placement_up (
    struct sprite_placement * sprite_placement )
```

Raise a sprite one level towards the top layer.

Parameters

<i>sprite_placement</i>	a handle to the placement
-------------------------	---------------------------

1.1.2.15 `animate_set_animation_function()`

```
void animate_set_animation_function (
    struct sprite_placement * sprite_placement,
    animate_fn ,
    void * priv )
```

@breif [Optional extension] define animation using the provided function pointer

Parameters

<i>sprite_placement</i>	a handle to the sprite placement
<i>animate_fn</i>	a custom function to call when determining the position of the sprite in each frame.
<i>priv</i>	private data that is passed to the provided function on call.
<i>ax</i>	the acceleration in the x direction
<i>ay</i>	the acceleration in the y direction

1.1.2.16 `animate_set_animation_params()`

```
void animate_set_animation_params (
    struct sprite_placement * sprite_placement,
    ssize_t vx,
    ssize_t vy,
    ssize_t ax,
    ssize_t ay )
```

Configure sprite animation.

Currently, we support only physics simulation (velocity and acceleration)

Parameters

<i>sprite_placement</i>	a handle to the sprite placement
<i>vx</i>	the initial velocity in the x direction
<i>vy</i>	the initial velocity in the y direction
<i>ax</i>	the acceleration in the x direction
<i>ay</i>	the acceleration in the y direction

1.2 `animate.h`

[Go to the documentation of this file.](#)

```
00001 #ifndef ANIMATE_H
00002 #define ANIMATE_H
00003
00004 #include <stdbool.h> /* for bool */
00005 #include <stdint.h> /* for uint32_t */
00006 #include <sys/types.h> /* for size_t/ssize_t */
00007
00015 struct sprite;
00016 struct canvas;
00017 struct sprite_placement;
00018
00022 typedef uint32_t color_t;
00023
00035 typedef void (*animate_fn)(void* priv, ssize_t* x, ssize_t* y, float t);
00036
00037 // Inline functions are often preferred over macros as types will be checked
00041 static inline color_t animate_color_rgb(unsigned r, unsigned g, unsigned b) {
00042     return ((r & 0xff) << 16) | ((g & 0xff) << 8) | ((b & 0xff) << 0);
00043 }
00044
00048 static inline color_t animate_color_argb(unsigned a,
00049     unsigned r, unsigned g, unsigned b) {
00050     return animate_color_rgb(r, g, b) | ((a & 0xff) << 24);
00051 }
00052
00063 struct canvas* animate_create_canvas(size_t height, size_t width,
00064     color_t background_color);
00065
00073 void animate_destroy_canvas(struct canvas* canvas);
00074
00086 struct sprite* animate_create_sprite(const char* file);
00087
00098 struct sprite* animate_create_rectangle(size_t width, size_t height, color_t c,
00099     bool filled);
00100
00117 struct sprite* animate_create_circle(size_t radius, color_t c, bool filled);
00118
00126 bool animate_destroy_sprite(struct sprite* sprite);
00127
00139 struct sprite_placement* animate_place_sprite(struct canvas* canvas,
00140     struct sprite* sprite,
00141     ssize_t x, ssize_t y);
00142
```

```
00148 void animate_placement_up(struct sprite_placement* sprite_placement);
00149
00155 void animate_placement_down(struct sprite_placement* sprite_placement);
00156
00162 void animate_placement_top(struct sprite_placement* sprite_placement);
00163
00169 void animate_placement_bottom(struct sprite_placement* sprite_placement);
00170
00176 void animate_destroy_placement(struct sprite_placement* sprite_placement);
00177
00178
00189 void animate_set_animation_params(struct sprite_placement* sprite_placement,
00190                                   ssize_t vx, ssize_t vy,
00191                                   ssize_t ax, ssize_t ay);
00192
00204 void animate_set_animation_function(struct sprite_placement* sprite_placement,
00205                                     animate_fn, void* priv);
00206
00215 size_t animate_frame_size_bytes(struct canvas* canvas);
00216
00227 void animate_generate_frame(const struct canvas* canvas,
00228                             size_t frame, size_t frame_rate,
00229                             void* buf);
00230
00231
00232 #endif /* ANIMATE_H */
```

